

# PRD-04

## Project Hub & Team Collaboration Backend

Version: 1.0

---

### Objective

Build a complete backend for project-based learning where students can browse projects, create teams, collaborate, assign tasks, upload documents, track milestones, and manage project progress.

The architecture should support future real-time collaboration without redesigning the database.

---

### Scope

This PRD includes:

- Project Catalog
  - Project Categories
  - Project Difficulty
  - Project Technologies
  - Team Management
  - Team Invitations
  - Roles
  - Task Management
  - Milestones
  - File Attachments
  - Project Progress
  - Comments
  - Activity Timeline
- 

### Database Models

---

## ProjectCategory

Examples:

- Web Development
- Mobile Apps
- AI/ML
- Cyber Security
- IoT
- Cloud
- DevOps

Fields:

- id
  - name
  - slug
  - description
  - icon
  - displayOrder
  - isActive
- 

## Project

Fields:

- id
  - categoryId
  - title
  - slug
  - description
  - overview
  - difficulty
  - estimatedDuration
  - thumbnail
  - githubRepository
  - liveDemo
  - documentationUrl
  - requirements
  - learningOutcomes
  - isPublished
  - createdAt
  - updatedAt
-

# ProjectTechnology

Fields:

- id
- name
- slug
- icon

Examples:

- React
  - Node.js
  - Express
  - Flutter
  - FastAPI
  - PostgreSQL
  - MongoDB
  - Docker
- 

## ProjectTechnologyRelation

Many-to-many.

- projectId
  - technologyId
- 

## Team

Fields:

- id
- projectId
- name
- ownerId
- maxMembers
- status
- createdAt

Status:

- OPEN
- FULL
- CLOSED
- COMPLETED

---

## TeamMember

Fields:

- id
- teamId
- userId
- role
- joinedAt

Roles:

- Owner
  - Leader
  - Developer
  - Designer
  - Researcher
  - Tester
- 

## TeamInvitation

Fields:

- id
- teamId
- senderId
- receiverId
- status
- expiresAt

Status:

- Pending
  - Accepted
  - Rejected
  - Expired
- 

## Task

Fields:

- id
- teamId
- assignedTo

- title
- description
- priority
- status
- dueDate
- estimatedHours
- createdAt

Status:

- Todo
- In Progress
- Review
- Completed

Priority:

- Low
  - Medium
  - High
  - Critical
- 

## Milestone

Fields:

- id
  - projectId
  - title
  - description
  - dueDate
  - status
  - completionPercentage
- 

## ProjectFile

Fields:

- id
- projectId
- uploadedBy
- title
- fileUrl
- fileType
- uploadedAt

Store files using **Supabase Storage**.

---

## TeamComment

Fields:

- id
  - taskId
  - userId
  - content
  - createdAt
- 

## ActivityLog

Track:

- Team created
- Member joined
- Task assigned
- Task completed
- Milestone completed
- File uploaded
- Comment added

Fields:

- id
  - teamId
  - userId
  - action
  - metadata (JSON)
  - createdAt
- 

## APIs

### Project Categories

GET /api/project-categories  
GET /api/project-categories/:id  
POST /api/project-categories  
PUT /api/project-categories/:id  
DELETE /api/project-categories/:id

---

## Projects

GET /api/projects  
GET /api/projects/:id  
POST /api/projects  
PUT /api/projects/:id  
DELETE /api/projects/:id

Support filtering by:

- Category
- Difficulty
- Technology
- Search
- Published

Pagination required.

---

## Teams

POST /api/teams  
GET /api/teams/:id  
PUT /api/teams/:id  
DELETE /api/teams/:id  
GET /api/my-teams

---

## Invitations

POST /api/team-invitations  
GET /api/team-invitations  
PATCH /api/team-invitations/:id

---

## Members

POST /api/teams/:id/members  
DELETE /api/teams/:id/members/:memberId  
PATCH /api/teams/:id/members/:memberId

---

## Tasks

GET /api/tasks  
POST /api/tasks  
PUT /api/tasks/:id  
DELETE /api/tasks/:id

---

## Milestones

GET /api/milestones  
POST /api/milestones  
PUT /api/milestones/:id  
DELETE /api/milestones/:id

---

## Files

POST /api/project-files  
GET /api/project-files/:projectId  
DELETE /api/project-files/:id

Upload files to **Supabase Storage** and persist metadata.

---

## Comments

POST /api/tasks/:id/comments  
GET /api/tasks/:id/comments  
PUT /api/comments/:id  
DELETE /api/comments/:id

---

## Activity

GET /api/teams/:id/activity

Return activity timeline in descending order.

---

## Dashboard APIs

GET /api/dashboard/projects

Return:

- Active Teams
  - Assigned Tasks
  - Completed Tasks
  - Upcoming Deadlines
  - Recent Activity
  - Team Invitations
-



# Business Rules

- One team belongs to one project.
  - A student cannot join the same team twice.
  - Team size cannot exceed `maxMembers`.
  - Only owners/leaders can invite members.
  - Only owners can delete a team.
  - Completed tasks are immutable unless reopened by the owner or leader.
  - Invitation links expire automatically.
  - Every task update creates an activity log entry.
  - Every file upload creates an activity log entry.
  - Deleting a team cascades to tasks, comments, and activity logs.
  - Duplicate technology relations are not allowed.
- 

# Validation

Use Zod for:

- Team size limits
  - Future due dates
  - Priority values
  - Status enums
  - URL validation (GitHub, Live Demo, Documentation)
  - File metadata
  - Slug uniqueness
- 

# Authorization

## Student

- Browse projects
- Create teams
- Join teams
- Manage own tasks
- Upload files
- Comment

## Team Leader

- Invite members
- Assign tasks
- Manage milestones

- Update team status

## Admin

- CRUD for project catalog
  - Moderate teams
  - Remove inappropriate content
  - Manage categories and technologies
- 

# Performance Requirements

- Pagination on all list endpoints.
  - Index:
    - projectId
    - teamId
    - assignedTo
    - ownerId
    - status
    - dueDate
  - Avoid N+1 queries.
  - Efficient relation loading with Prisma.
- 

# API Response Standard

Continue using the standardized response format from PRD-01.

---

# Acceptance Criteria

PRD-04 is complete when:

- All new Prisma models and relationships are implemented.
- Migrations execute successfully.
- CRUD APIs for project categories, projects, teams, tasks, milestones, files, and comments are functional.
- Team invitations, membership management, and role permissions work correctly.
- Activity logs are automatically generated for tracked actions.
- Dashboard endpoints return aggregated project data.
- Pagination, filtering, sorting, validation, and authorization are implemented.
- Supabase Storage integration works for project files.
- Swagger documentation is updated.
- Logging and error handling remain consistent with PRD-01.

---

## Architectural recommendation before moving to PRD-05

Introduce a reusable **Notification/Event layer** now, even if you don't expose notification APIs yet.

Instead of sending notifications directly inside services (e.g., "task assigned"), emit domain events such as:

- TaskAssigned
- InvitationAccepted
- MilestoneCompleted
- ProjectCreated

Then have listeners handle side effects like notifications, emails, analytics, or future WebSocket updates.

This event-driven approach keeps your services focused on business logic and makes it much easier to add real-time features and notifications in later PRDs without rewriting existing modules.